



MCDIA500 • PROGRAMACIÓN PARA LA CIENCIA DE DATOS
MAGÍSTER EN CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL
UNIVERSIDAD ANDRÉS BELLO

Análisis de ofertas en licitaciones públicas del sector Salud

Sumativa 1: Avance del proyecto Fases 1 y 2 – Entorno reproducible, exploración y procesamiento inicial de datos

Reporte oficial de marzo de 2026

Grupo 5

Víctor Bravo Barrera

Nayadeth Garrido Ibáñez

Mauricio Cid

Docente: Omar Salinas Silva

Fecha: 16 de septiembre de 2026

Índice de contenidos

I. Introducción y contextualización.....	3
Conexión explícita con las fases F1–F4 del proyecto	3
II. Definición de la problemática y objetivos del proyecto	5
Preguntas específicas del estudio.....	5
Objetivos del proyecto.....	5
Alcance, restricciones y supuestos analíticos	5
III. Herramientas científicas, reproducibilidad y control de versiones	7
Arquitectura modular y estructura del repositorio	7
Gestión colaborativa y control de versiones.....	8
IV. Diseño de soluciones algorítmicas, preprocesamiento y validación (F2)	9
A. Codificación funcional modular	9
B. Preprocesamiento, transformaciones y tratamiento de NA.....	9
C. Validación técnica y verificación del código.....	11
V. Resultados preliminares de la preparación de datos (F1 y F2).....	11
Proporción de ofertas ganadoras según tamaño de proveedor	11
Distribución por tipo de licitación y plazos administrativos	12
VI. Evidencias de ejecución de las Fases 1 y 2.....	12
Evidencia del Notebook F1_Definición.ipynb	13
Evidencia del Notebook F2_Preprocesamiento.ipynb	14
VII. Vinculación del mapa conceptual con el avance del proyecto	18
VIII. Referencias bibliográficas (Normas APA 7.ª edición).....	20
IX. Anexos técnicos y trazabilidad	21

I. Introducción y contextualización

Las compras públicas en Chile constituyen uno de los mecanismos más relevantes de asignación de recursos fiscales, donde el sector Salud representa un volumen crítico de contrataciones de insumos, equipamiento y servicios clínicos esenciales (ChileCompra, s. f.-a). A través de la plataforma Mercado Público, gestionada por la Dirección de Compras y Contratación Pública (ChileCompra), los organismos de salud publican sus requerimientos y reciben propuestas de múltiples proveedores bajo diversas modalidades de licitación.

El presente estudio analiza el reporte de contrataciones del sector Salud correspondiente a marzo de 2026, el cual reúne 44.226 registros y 74 columnas. Una característica estructural fundamental de este conjunto de datos, señalada por ChileCompra (s. f.-a), es que cada fila representa una oferta individual asociada a un ítem específico dentro de un proceso de licitación. En consecuencia, el número de observaciones no equivale a la cantidad de licitaciones únicas: un mismo proceso licitatorio (`NroLicitacion`) puede albergar múltiples productos o líneas de cotización y recibir propuestas concurrentes de diversos oferentes. La interpretación inicial de los campos se apoyó también en el diccionario de la API y en el glosario de datos abiertos de ChileCompra, considerando que estas fuentes no reproducen exactamente el esquema del reporte analizado (ChileCompra, s. f.-b, s. f.-c).

El problema central consiste en describir cómo varía la proporción de ofertas ganadoras según el tipo de licitación y el tamaño del proveedor en las adquisiciones de salud. El volumen de registros y la combinación de variables categóricas, monetarias y temporales requieren un tratamiento programático con Python, NumPy y pandas, junto con prácticas de reproducibilidad computacional (Samuel & Mietchen, 2024).

En este contexto, el **objetivo general del proyecto** es analizar descriptivamente las diferencias en la proporción de ofertas ganadoras según el tipo de licitación y el tamaño del proveedor en el sector Salud durante marzo de 2026, mediante un flujo de programación científica, preparación de datos y análisis reproducible. En cuanto al **alcance de esta entrega**, el trabajo se concentra en las Fases 1 y 2 del proyecto, abordando la definición de la problemática y sus objetivos, la configuración del entorno reproducible, la exploración inicial del conjunto de datos, la identificación y tratamiento de problemas de calidad, la transformación de variables y la validación técnica del dataset resultante. Los análisis estadísticos bivariados y multivariados, las visualizaciones analíticas avanzadas y la integración de conclusiones finales quedan proyectados para las Fases 3 y 4.

Conexión explícita con las fases F1–F4 del proyecto

El proyecto se articula en una progresión metodológica de cuatro fases interdependientes:

Fase 1 (Implementada): Delimitación de la problemática, preguntas de análisis, objetivos, supuestos teóricos, diseño de la arquitectura modular y configuración del entorno reproducible verificado.

Fase 2 (Implementada): Diagnóstico exploratorio de calidad de datos, desarrollo del pipeline modular de limpieza (exclusión de columnas nulas y neutralización de fechas anómalas año 1900), transformaciones de tipos, validación técnica automatizada y exportación del dataset preprocesado a data/processed/.

Fase 3 (Proyectada): Análisis exploratorio bivariado y multivariado, modelación estadística de tasas de adjudicación y visualización analítica avanzada con librerías científicas de Python.

Fase 4 (Proyectada): Consolidación de hallazgos, evaluación del impacto de las políticas de compra pública en el sector salud, discusión de limitaciones y formulación de recomendaciones finales.

II. Definición de la problemática y objetivos del proyecto

La adquisición pública de bienes y servicios médicos exige maximizar la transparencia y la libre concurrencia de proveedores. En este contexto, interesa describir si la proporción de ofertas ganadoras cambia según el tamaño de la empresa oferente (TamanoProveedor: Micro, Pequeña, Mediana, Grande o NoClasificado) y el tipo de licitación registrado en el conjunto de datos.

Pregunta central de investigación: ¿Cómo varía la proporción de ofertas registradas como ganadoras según el tipo de licitación y el tamaño del proveedor en el reporte del sector Salud de marzo de 2026?

Preguntas específicas del estudio

¿Cuál es la distribución de ofertas e ítems licitados entre las diferentes categorías de tamaño de proveedor y tipos de licitación pública?

¿Qué porcentaje de ofertas resulta efectivamente adjudicado dentro de cada estrato empresarial y cómo influye el estado administrativo del proceso (adjudicada vs. desierta, revocada o cerrada)?

¿Qué inconsistencias estructurales, columnas vacías o codificaciones anómalas (fechas en 1900, categorías no clasificadas) condicionan la validez de las métricas de éxito en los datos de compras públicas?

Objetivos del proyecto

Objetivo general: Analizar descriptivamente las diferencias en la proporción de ofertas ganadoras según el tipo de licitación y el tamaño del proveedor en el sector Salud durante marzo de 2026, mediante un flujo reproducible de preparación y validación de datos.

Objetivos Específicos:

1. Delimitar conceptualmente la unidad de análisis, variables clave, restricciones y configurar un entorno reproducible trazable bajo control de versiones (F1).
2. Caracterizar la estructura de las 74 variables, diseñar e implementar un pipeline modular de limpieza y transformación de datos, y verificar su robustez técnica mediante pruebas de casos límite y excepciones (F2).
3. Construir distribuciones, cruces de contingencia y visualizaciones efectivas que revelen la brecha de éxito entre micro, pequeñas, medianas y grandes empresas (F3).
4. Integrar los resultados técnicos, evidencias de ejecución y análisis crítico en un reporte formal orientado a la toma de decisiones en políticas públicas de abastecimiento sanitario (F4).

Alcance, restricciones y supuestos analíticos

El análisis se define bajo los siguientes supuestos y límites metodológicos:

Enfoque descriptivo: Las asociaciones encontradas describen comportamientos del periodo observado y no implican causalidad ni juicio sobre la eficiencia contractual.

Cobertura temporal y geográfica: El archivo comprende ofertas del reporte de marzo de 2026

del sector Salud chileno, con publicaciones ocurridas entre el 15 de enero y el 25 de marzo de 2026 y cierres concentrados en marzo de 2026.

Granularidad de fila: Cada observación representa una oferta para una línea de producto; por ende, varias filas pueden corresponder a una misma licitación y proveedor.

Proporción de ofertas ganadoras: se calcula como el número de ofertas ganadoras dividido por el total de ofertas con resultado válido en procesos adjudicados. El denominador se informa en cada comparación.

III. Herramientas científicas, reproducibilidad y control de versiones

Para reproducir los resultados se configuró un entorno virtual (.venv) con Python 3.12.14 en Windows x64 (Samuel & Mietchen, 2024; Universidad Andrés Bello, 2026b).

Herramienta / Librería	Versión verificada	Función en el proyecto
Python	3.12.14	Intérprete base de ejecución estructurada y scripting modular.
pandas	3.0.1	Manipulación tabular, preprocesamiento, casting datetime y cálculo de frecuencias.
NumPy	2.3.5	Operaciones vectorizadas, manejo de valores nulos (np.nan) y aserciones numéricas.
JupyterLab / ipykernel	4.6.3 / 7.3.0	Entorno interactivo reproducible que integra narrativa técnica, código y salidas.
Git / GitHub	2.x	Sistema distribuido de control de versiones y trazabilidad colaborativa.
nbclient / nbformat	0.11.0 / 5.11.1	Automatización de ejecución desatendida y verificación técnica de notebooks.

Arquitectura modular y estructura del repositorio

El proyecto adopta el principio de separación de responsabilidades (McKinney, 2022). La lógica computacional reutilizable reside en el módulo `src/proyecto.py`, mientras que los cuadernos interactivos `F1/F1_Definición.ipynb` y `F2/F2_Preprocesamiento.ipynb` integran explicación, código y resultados (Project Jupyter, s. f.):

`src/proyecto.py`: Contiene funciones reutilizables para registrar versiones, calcular huellas SHA-256, leer datos con un contrato de esquema, diagnosticar su calidad, ejecutar el pipeline de limpieza, codificar variables nominales y validar los resultados.

`data/raw/`: Almacena el dataset original `licitaciones_salud_marzo_2026.csv` (68,96 MB) con hash inmutable SHA-256.

`data/processed/`: Aloja el producto derivado `licitaciones_salud_marzo_2026_procesado.csv` listo para análisis analítico.

`evidencias/`: Conserva los registros estructurados JSON (`F1_ejecucion.json` y `F2_ejecucion.json`) emitidos tras ejecuciones exitosas.

`docs/`: Reúne el informe integrado en Word/PDF y el modelo conceptual técnico editable en `draw.io`.

Gestión colaborativa y control de versiones

El desarrollo se gestionó en GitHub, en el repositorio público <https://github.com/vbravo29/sumativo-1> y sobre la rama activa desarrollo. Los cambios se integraron mediante ramas y pull requests, manteniendo un historial con aportes verificables de cada integrante del Grupo 5 (GitHub, s. f.):

Commit	Autor registrado	Hito técnico alcanzado
03bf771	Víctor Bravo Barrera (vbravo29)	Mejoras de redacción y consistencia general del informe técnico.
57ccca6	Víctor Bravo Barrera (vbravo29)	Implementación de one-hot encoding, refactorización de métodos bajo principio DRY y mejoras de documentación.
ef00f72	Nayadeth Garrido Ibáñez	Corrección general del formato del informe e incorporación de versión descargable del mapa conceptual.
ecd66a2	Víctor Bravo Barrera (vbravo29)	Mejoras en la estructura y contenido del mapa conceptual técnico.
78ee9e8	Víctor Bravo Barrera (vbravo29)	Integración de cambios de la rama desarrollo y sincronización del trabajo colaborativo.
eef148a	Nayadeth Garrido Ibáñez	Avance y actualización del contenido del informe técnico.
eed767	Víctor Bravo Barrera (vbravo29)	Modificaciones al mapa conceptual para mejorar su correspondencia con la implementación del proyecto.
cb49a04	Víctor Bravo Barrera (vbravo29)	Mejoramiento del diagrama y del flujo de trabajo representado en el mapa conceptual.
b213be1	Víctor Bravo Barrera (vbravo29)	Correcciones finales de estructura y contenido del mapa conceptual.
116d2bc	Nayadeth Garrido Ibáñez	Mejoramiento del informe e incorporación de evidencia de ejecución de Jupyter Notebook.
ee2f37f	Nayadeth Garrido (garridonayadeth-byte)	Integración mediante Pull Request #6 de los cambios desarrollados en la rama informe.
96bffc1	Nayadeth Garrido Ibáñez	Incorporación de evidencias de la Fase 2 y mejoramiento de la introducción del informe.
a5b6516	Nayadeth Garrido (garridonayadeth-byte)	Integración mediante Pull Request #7 de nuevas modificaciones realizadas en la rama informe.
60b4846	Víctor Bravo Barrera (vbravo29)	Actualización del informe con la versión vigente del mapa conceptual.

Commit	Autor registrado	Hito técnico alcanzado
6fc31fe	Nayadeth Garrido Ibáñez	Actualización de la evidencia del historial de commits en el informe.
13fc069	Mauricio Cid	Registro de decisiones técnicas y alternativas evaluadas en las Fases 1 y 2.
fcc770	Mauricio Cid	Actualización del mapa conceptual con el flujo reproducible, las decisiones del equipo y el glosario visual.

IV. Diseño de soluciones algorítmicas, preprocesamiento y validación (F2)

Esta sección detalla el desarrollo técnico de la Fase 2, evidenciando la construcción de scripts funcionales, el tratamiento justificado de valores faltantes y la ejecución de pruebas sistemáticas sobre los datos.

A. Codificación funcional modular

La ingeniería de datos se implementó a través de funciones estructuradas en `src/proyecto.py` con parámetros explícitos, manejo de excepciones y validación de precondiciones:

`resumen_exploracion(df)`: Computa el inventario de filas, columnas, variables sin nulos, columnas con nulos parciales, columnas 100% vacías y campos constantes.

`limpiar_datos_f2(df, excluir_vacias=True, normalizar_texto=True)`: Ejecuta las transformaciones de limpieza sobre una copia profunda del DataFrame, asegurando la inmutabilidad de la entrada.

`validar_dataset_procesado(df, filas_esperadas)`: Ejecuta seis grupos de comprobaciones sobre columnas vacías, cantidad de filas, variables obligatorias, resultados válidos, fechas anómalas y columnas derivadas.

`exportar_datos_procesados(df, ruta_salida)`: Serializa el dataset limpio a CSV e informa su peso en MB y su huella criptográfica SHA-256.

`codificar_nominales(df, columnas)` y `validar_codificacion(original, codificado, columnas)`: Generan indicadores binarios para las variables nominales y comprueban la correspondencia entre categorías, filas y columnas resultantes.

B. Preprocesamiento, transformaciones y tratamiento de NA

El análisis exploratorio de las 74 variables iniciales reveló anomalías estructurales que fueron intervenidas con criterios explícitos y reproducibles:

Variable Componente	Problema detectado	Tratamiento aplicado en F2	Justificación metodológica
LicitacionBaseTipo, ContratoRenovable , UnidadTiempoRenovacion	100% de valores nulos (44.226 filas vacías).	Exclusión directa del conjunto de datos.	No aportan variabilidad ni información a ninguna etapa del análisis.
FechaEstimadaEvaluacionOfertas	7.613 filas registran fechas del año 1900 (1900-01-11, etc.).	Conversión sistemática a NaT.	Las fechas del año 1900 no son coherentes con el período del reporte; conservarlas distorsionaría el cálculo de plazos.
FechaPublicacion, FechaCierre, FechaAdjudicacion	Almacenadas como cadenas de texto (str).	Casting datetime64[ns] mediante pd.to_datetime.	Habilita operaciones temporales y cómputo de plazos administrativos.
TamanoProveedor	3.410 ofertas registran la etiqueta NoClasificado.	Preservación explícita como categoría separada.	Imputar arbitrariamente distorsionaría la representación real de empresas en compras públicas.
ResultadoOferta vs. EstadoLicitacion	Ofertas en licitaciones desiertas o revocadas figuran como perdedoras.	Creación de flags licitacion_adjudicada y oferta_ganadora.	Permite calcular proporciones de ofertas ganadoras únicamente dentro de procesos adjudicados.
plazo_cierre_dias	Variable no existente en el CSV original.	Cálculo (FechaCierre FechaPublicacion) en días.	Permite describir el tiempo registrado entre la publicación y el cierre de recepción de ofertas.
TipoLicitacion y TamanoProveedor	Variables nominales con categorías	Codificación one-hot en un dataset adicional,	Genera indicadores 0/1 sin imponer un orden numérico artificial entre

Variable Componente	Problema detectado	Tratamiento aplicado en F2	Justificación metodológica
	textuales.	conservando las variables originales.	categorías.

No se aplicó imputación estadística a los valores faltantes. Las columnas completamente vacías fueron excluidas, mientras que los valores faltantes parciales se conservaron para evitar introducir información artificial. La categoría NoClasificado se mantuvo como una categoría explícita. Para preparar las variables nominales se aplicó codificación one-hot a TipoLicitacion y TamanoProveedor, generando indicadores binarios sin asignar un orden numérico artificial a sus categorías. Las variables originales se conservaron y el resultado codificado se exportó en un archivo separado.

C. Validación técnica y verificación del código

El notebook F2_Preprocesamiento.ipynb ejecuta pruebas automáticas sobre los siguientes escenarios:

Caso de prueba	Escenario evaluado	Comportamiento esperado	Resultado
Caso Normal	Dataset real de 44.226 filas preprocesado.	Superar 6 aserciones de consistencia e integridad.	OK (6/6 superadas)
Caso límite	Registros con fechas año 1900 y categoría NoClasificado.	Fecha convertida a NaT y categoría preservada.	OK (Neutralización exitosa)
Caso Excepción 1	DataFrame sin la columna obligatoria ResultadoOferta.	Captura controlada de excepción KeyError.	OK (KeyError capturado)
Caso Excepción 2	DataFrame vacío (0 registros).	Captura controlada de excepción ValueError.	OK (ValueError capturado)

V. Resultados preliminares de la preparación de datos (F1 y F2)

Tras ejecutar el pipeline sobre el conjunto completo de licitaciones del sector Salud, se obtuvieron resultados descriptivos que permiten comprobar la preparación del dataset y orientar los análisis de las fases posteriores.

Proporción de ofertas ganadoras según tamaño de proveedor

Al restringir el análisis a procesos adjudicados (44.025 ofertas), se observan diferencias

descriptivas entre los tamaños de proveedor:

Tamaño del proveedor	Ofertas ganadoras	Ofertas perdedoras	Total de ofertas	% de ofertas ganadoras dentro del tamaño
Grande	12.575	6.255	18.830	66,78 %
Mediana	4.982	4.001	8.983	55,46 %
NoClasificado	1.803	1.576	3.379	53,36 %
Pequeña	5.022	4.577	9.599	52,32 %
Micro	1.680	1.554	3.234	51,95 %
Total (procesos adjudicados)	26.062	17.963	44.025	59,20 %

Nota. Cada porcentaje se calcula de manera independiente dentro de cada categoría de tamaño del proveedor. En consecuencia, estos valores no representan partes de una misma distribución y su suma no corresponde al 100 %.

Las empresas de tamaño Grande presentan una proporción de ofertas ganadoras de 66,78 %, superior en 14,83 puntos porcentuales a las Microempresas (51,95 %) y en 14,46 puntos porcentuales a las Pequeñas empresas (52,32 %). Estas diferencias describen los registros del período y no establecen causalidad. Su interpretación se profundizará en las fases posteriores.

Distribución por tipo de licitación y plazos administrativos

El tipo de licitación más frecuente corresponde a Licitación Pública Entre 100 y 1000 UTM (LE), con 23.708 ofertas, seguida de Licitación Pública Mayor 1000 UTM (LP), con 14.448 ofertas. El plazo promedio registrado entre la publicación y el cierre de recepción fue de 14,38 días (desviación estándar de 6,86 días), con un mínimo de 4,92 y un máximo de 61,03 días. Estos valores describen el conjunto de datos y no permiten concluir por sí solos el cumplimiento normativo, ya que los plazos aplicables dependen del tipo de licitación, el monto y las condiciones particulares de cada proceso.

VI. Evidencias de ejecución de las Fases 1 y 2

Evidencia del Notebook F1_Definición.ipynb

El notebook F1/F1_Definición.ipynb documenta e implementa la Fase 1 del proyecto, incorporando la problemática, los objetivos, la definición de la unidad de análisis y la configuración inicial del entorno reproducible. El cuaderno utiliza las funciones reutilizables definidas en src/proyecto.py y mantiene correspondencia con la planificación establecida en el mapa conceptual técnico. Su ejecución fue verificada bajo Python 3.12, registrándose 4 celdas ejecutadas y 0 errores, según la evidencia almacenada en evidencias/F1_ejecucion.json.

6. Herramientas y entorno

Python organiza el flujo mediante funciones; pandas permite leer el CSV; NumPy se comprueba con una operación sintética; Jupyter integra narrativa, código y salidas. Git registra la evolución del proyecto. La prueba de NumPy usa `arange` para construir una secuencia entera verificable (NumPy Developers, s. f.).

Las instrucciones de instalación están en el [README](#) y las dependencias principales en `requirements.txt`. Las versiones utilizadas y la versión efectiva de Python se registran abajo. La referencia previa a Python 3.14.7 no demuestra compatibilidad; esta revisión se verifica en Python 3.12 y cualquier otro entorno requiere su propia ejecución completa.

```
from pathlib import Path
import sys
import platform

# La estructura identifica la raíz incluso al abrir el notebook desde F1.
raiz = next((p for p in (Path.cwd(), *Path.cwd().parents)
             if (p / "src" / "proyecto.py").is_file() and (p / "F1").is_dir()), None)
if raiz is None:
    raise FileNotFoundError("Abrir el notebook desde la raíz del repositorio o F1.")
if str(raiz) not in sys.path:
    sys.path.insert(0, str(raiz))

import numpy as np
import pandas as pd
from src.proyecto import versiones_entorno, sha256_archivo, leer_datos_f1

entorno = {"python": platform.python_version(), "sistema": platform.platform(),
           "entorno_virtual": sys.prefix != sys.base_prefix,
           "dependencias": versiones_entorno()}
display(entorno)
muestra = pd.DataFrame({"valor": np.arange(3)})
assert muestra["valor"].sum() == 3
print("OK: Integración mínima de NumPy y pandas.")
```

```
{'python': '3.12.14',
 'sistema': 'Windows-11-10.0.26200-SP0',
 'entorno_virtual': True,
 'dependencias': {'numpy': '2.3.5',
                  'pandas': '3.0.1',
                  'sha256': '44226'}}
```

7. Lectura inicial y trazabilidad del dataset

Se verifica la huella SHA-256 documentada antes de leer el archivo. Las dimensiones esperadas describen esta versión concreta del CSV: un cambio exige revisar la procedencia y actualizar la documentación, no ignorar la comprobación. La lectura conserva las filas y columnas y no genera datos procesados.

```
ruta_datos = raiz / "data" / "raw" / "licitaciones_salud_marzo_2026.csv"
columnas_requeridas = ["NroLicitacion", "TipoLicitacion", "TamanoProveedor",
                       "ResultadoOferta", "Sector", "FechaPublicacion"]
huella_esperada = "490d209a10d387011d481b72b7891f26e997974ec2cf9dfc518aa4a08552232"
huella = sha256_archivo(ruta_datos)
assert huella == huella_esperada, "El CSV difiere de la versión documentada."
datos = leer_datos_f1(ruta_datos, columnas_requeridas)
assert datos.shape == (44226, 74), "Revisar dimensiones de la versión de entrada."
resumen_entrada = {"archivo": ruta_datos.name, "filas": len(datos),
                   "columnas": len(datos.columns), "sha256": huella,
                   "columnas_requeridas_presentes": True}
display(resumen_entrada)
display(pd.DataFrame({"variable": columnas_requeridas,
                     "tipo_inferido": [str(datos[c].dtype) for c in columnas_requeridas]}))
print("OK: entrada disponible, íntegra y con el esquema mínimo esperado.")
```

```
{'archivo': 'licitaciones_salud_marzo_2026.csv',
 'filas': 44226,
 'columnas': 74,
 'sha256': '490d209a10d387011d481b72b7891f26e997974ec2cf9dfc518aa4a08552232',
 'columnas_requeridas_presentes': True}
```

	variable	tipo_inferido
0	NroLicitacion	str
1	TipoLicitacion	str
2	TamanoProveedor	str
3	ResultadoOferta	str
4	Sector	str
5	FechaPublicacion	str

8. Verificación del código inicial

Las pruebas siguientes comprueban el contrato de lectura con entradas pequeñas: caso normal, ausencia de archivo, ausencia de una columna y archivo con encabezado pero sin registros. No son pruebas del futuro pipeline F2. Los archivos de prueba se crean en una carpeta temporal que se elimina al terminar.

```
from tempfile import TemporaryDirectory

pruebas = []
with TemporaryDirectory() as temporal:
    carpeta = Path(temporal)
    normal = carpeta / "normal.csv"
    normal.write_text("Tipolicitacion;ResultadoOferta;Almacenamiento", encoding="latin-1")
    resultado = leer_datos_f1(normal, [{"Tipolicitacion": "ResultadoOferta"}])
    assert resultado.shape == (1, 3) and resultado.loc[0, "ResultadoOferta"] == "Sanadora"
    pruebas.append({"caso": "Lectura normal", "resultado": "OK"})

    vacio = carpeta / "vacio.csv"
    vacio.write_text("Tipolicitacion;ResultadoOferta;n", encoding="latin-1")
    casos = [
        ("Archivo ausente", carpeta / "ausente.csv", [{"Tipolicitacion": "ResultadoOferta"}], FileNotFoundError),
        ("Columna ausente", normal, [{"Almacenamiento": "ResultadoOferta"}], ValueError),
        ("Sin registros", vacio, [{"Tipolicitacion": "ResultadoOferta"}], ValueError),
    ]
    for nombre, ruta, columnas, excepcion in casos:
        try:
            leer_datos_f1(ruta, columnas)
        except excepcion as error:
            pruebas.append({"caso": nombre, "resultado": "OK", "detalle": str(error)})
        else:
            raise AssertionError(f"No se detectó el caso: {nombre}")
    display(pd.DataFrame(pruebas))
    assert sha256_archivo(ruta_datos) == huella, "El original fue modificado."
    print("OK: cuatro pruebas y conservación del archivo original.")
```

caso	resultado	detalle
0 Lectura normal	OK	NaN
1 Archivo ausente	OK	No existe un archivo de datos en C:\Users\lbr...
2 Columna ausente	OK	Faltan columnas requeridas: Almacenamiento
3 Sin registros	OK	El dataset no contiene registros.

OK: cuatro pruebas y conservación del archivo original.

Evidencia del Notebook F2_Preprocesamiento.ipynb

El notebook F2/F2_Preprocesamiento.ipynb implementa el flujo técnico correspondiente a la Fase 2, incluyendo exploración inicial, diagnóstico de calidad, limpieza, tratamiento de valores nulos, transformación de variables, generación de atributos derivados y validación del dataset resultante. Cada etapa se encuentra documentada mediante celdas narrativas y resultados verificables, manteniendo trazabilidad con las funciones implementadas en src/proyecto.py y con el conjunto de datos procesado generado en data/processed/. Su ejecución fue verificada bajo Python 3.12, registrándose 10 celdas ejecutadas y 0 errores, de acuerdo con evidencias/F2_ejecucion.json.

1. Configuración del entorno y funciones modulares

Se carga el entorno de trabajo y se importan las funciones implementadas en src/proyecto.py, garantizando la separación entre lógica de procesamiento y presentación interactiva.

```
from pathlib import Path
import sys
import numpy as np
import pandas as pd

raiz = next(p for p in (Path.cwd(), *Path.cwd().parents)
            if (p / "src" / "proyecto.py").is_file() and (p / "F2").is_dir(), None)
if raiz is None:
    raise FileNotFoundError("Abrir el notebook desde la raíz del repositorio o F2.")
if str(raiz) not in sys.path:
    sys.path.insert(0, str(raiz))

from src.proyecto import (
    versiones_entorno, sha256_archivo, leer_datos_f1,
    resumen_exploracion, limpiar_datos_f2, validar_dataset_procesado,
    exportar_datos_procesados
)

print("Directorio raíz del proyecto:", raiz)
display(versiones_entorno())
```

Directorio raíz del proyecto: C:\Trabajo\Unab\Unab-1

```
{'numpy': '2.3.5',
 'pandas': '3.0.1',
 'jupyterlab': '4.6.3',
 'ipykernel': '7.3.0',
 'nbformat': '5.11.1',
 'nbconvert': '7.17.1',
 'nbclient': '0.11.0'}
```

2. Obtención y verificación criptográfica de los datos de entrada

Se verifica la huella digital SHA-256 del dataset original licitaciones_salud_marzo_2026.csv antes de procesar para asegurar que el insumo no haya sido alterado.

```
ruta_raw = raíz / "data" / "raw" / "licitaciones_salud_marzo_2026.csv"
huella_esperada = "49b05209a3b387811681b372b7891f26e97974ec2cf96fc33baa4b852232"
huella_actual = sha256_path(ruta_raw)
assert huella_actual == huella_esperada, f"Error: la huella SHA-256 del CSV no coincide con la versión documentada."

columnas_esquema = ["TipoLicitacion", "TipoLicitacion", "TamanoProveedor", "ResultadoOferta", "EstadoLicitacion", "Sector", "FechaPublicacion"]
df_raw = leer_datos(ruta_raw, columnas_esquema)
print("Dataset de entrada cargado exitosamente:")
print(f"- Filas totales: {len(df_raw)}")
print(f"- Columnas totales: {len(df_raw.columns)}")
print(f"- SHA-256 verificado: {huella_actual}")
assert df_raw.shape == (44226, 74)
```

Dataset de entrada cargado exitosamente:

- Filas totales: 44226
- Columnas totales: 74
- SHA-256 verificado: 49b05209a3b387811681b372b7891f26e97974ec2cf96fc33baa4b852232

3. Exploración inicial (EDA) y diagnóstico de calidad

Mediante resumen_exploracion, analizamos la completitud del conjunto de datos, identificando columnas sin variabilidad o completamente nulas.

```
metricas_eda = resumen_exploracion(df_raw)
df_metricas = pd.DataFrame([
    ("Metrica": "Total de filas observadas", "Valor": metricas_eda["total_filas"]),
    ("Metrica": "Total de columnas", "Valor": metricas_eda["total_columnas"]),
    ("Metrica": "Columnas completas (sin nulos)", "Valor": metricas_eda["columnas_completas"]),
    ("Metrica": "Columnas con valores nulos parciales", "Valor": metricas_eda["columnas_con_nulos"]),
    ("Metrica": "Columnas 100% vacías", "Valor": len(metricas_eda["columnas_100_nulos"])),
    ("Metrica": "Columnas con valor único (constantes)", "Valor": len(metricas_eda["columnas_constantes"])),
])
display(df_metricas)
print("Columnas 100% vacías identificadas para exclusión:", metricas_eda["columnas_100_nulos"])
print("Columnas con valor único: ", metricas_eda["columnas_constantes"])
print("Duplicados exactos: ", len(df_raw.drop_duplicates().sum()))
print("Licitaciones distintas: ", df_raw.NroLicitacion.nunique())
```

Métrica Valor

Distribuciones de las variables clave del estudio

Examinamos la distribución de frecuencias de las variables principales (TipoLicitacion, TamanoProveedor, ResultadoOferta) y su interacción con EstadoLicitacion.

```
from src.proyecto import tablas_frecuencia

for columna, tabla in tablas_frecuencia(df_raw, ["TipoLicitacion", "TamanoProveedor", "ResultadoOferta"]).items():
    print("Distribución de ", columna)
    display(tabla)
    print(f"EstadoLicitacion frente a {columna}:")
    display(pd.crosstab(df_raw["EstadoLicitacion"], df_raw[columna], margins=True))
```

Distribución de TipoLicitacion

TipoLicitacion	Frecuencia
Licitación Pública Entre 100 y 1000 UTM (L1)	23708
Licitación Pública Mayor 1000 UTM (L2)	14448
Licitación Pública Mayor a 5000 (L3)	3227
Licitación Pública Menor a 100 UTM (L4)	2757
Licitación Privada Mayor a 1000 UTM	72
Licitación Privada entre 100 y 1000 UTM	42
Licitación Privada Mayor a 5000 (L5)	2

Distribución de TamanoProveedor

TamanoProveedor	Frecuencia
Grande	18865
Pequeña	9682
Mediana	9015
NoClasificado	3410
Micro	3254

Distribución de ResultadoOferta

ResultadoOferta	Frecuencia
Ganadora	28063
Perdedora	18163

EstadoLicitacion frente a ResultadoOferta:

4. Pipeline modular de limpieza y transformación de datos

Se ejecuta la función limpiar_datos_f2(df_raw) de src/proyecto.py. Sus etapas y justificaciones son:

- Exclusión de columnas 100% vacías:** Se descartan LicitacionBaseTipo, ContratoRenovable y UnidadTiempoRenovacion.
- Neutralización de fechas anómalas:** Los 7.613 valores de FechaEstimadaEvaluacionOfertas fijados en el año 1900 corresponden a fechas anómalas para el periodo estudiado, cuyo significado oficial no se ha confirmado; se convierten a NaT.
- Casting a datetime:** Las columnas de fecha se convierten a formato de fecha/hora de pandas (datetime64[ns]), permitiendo cálculos de plazos.
- Normalización de texto:** Se remueven espacios en blanco residuales (str.strip()).
- Preselección de categorías:** Se preserva la etiqueta NoClasificado en TamanoProveedor (3.410 ofertas), evitando imputaciones artificiales.
- Ingeniería de variables:**
 - oferta_ganadora: booleano (True si ResultadoOferta == 'Ganadora').
 - licitacion_adjudicada: booleano (True si EstadoLicitacion == 'Adjudicada').
 - plazo_cierre_dias: cálculo de días entre publicación y cierre del proceso. Los NA parciales se conservan sin imputar. Las fechas usadas se convierten explícitamente a datetime64[ns]; una fecha no interpretable detiene el proceso para revisión. Los demás campos temporales quedan fuera del cálculo actual. No se escala porque se calculan frecuencias y proporciones. No se eliminan filas por compartir NroLicitacion: un proceso reúne varios ítems y ofertas. La clave única de oferta/ítem sigue pendiente de contraste con el esquema del reporte.

```
df_procesado = limpiar_datos_f2(df_raw)
print("Dimensiones tras preprocesamiento:", df_procesado.shape)

cols_excluidas = ["LicitacionBaseTipo", "ContratoRenovable", "UnidadTiempoRenovacion"]
presentes = [c for c in cols_excluidas if c in df_procesado.columns]
assert len(presentes) == 0, f"Error: columnas vacías aún presentes: {presentes}"
print("OK: Columnas vacías excluidas correctamente.")

print("Tipo inferido FechaPublicacion:", df_procesado["FechaPublicacion"].dtype)
print("Tipo inferido FechaCierre:", df_procesado["FechaCierre"].dtype)
print("Tipo inferido FechaAdjudicacion:", df_procesado["FechaAdjudicacion"].dtype)

plazo_stats = df_procesado["plazo_cierre_dias"].describe()
print("Estadísticos de plazo_cierre_dias:")
display(plazo_stats.to_frame())
print("Fechas de evaluación faltantes tras limpieza:", int(df_procesado.FechaEstimadaEvaluacionOfertas.isna().sum()))
print("Los plazos anteriores están ponderados por registros de oferta, no por licitaciones únicas.")
```

Dimensiones tras preprocesamiento: (44226, 74)
OK: Columnas vacías excluidas correctamente.
Tipo inferido FechaPublicacion: datetime64[ns]
Tipo inferido FechaCierre: datetime64[ns]
Tipo inferido FechaAdjudicacion: datetime64[ns]

Estadísticos de plazo_cierre_dias:

plazo_cierre_dias	
count	44226.000000
mean	14.384128

5. Análisis exploratorio de proporciones de éxito

Con los datos limpios, comparamos la tasa de adjudicación según el tamaño del proveedor en licitaciones que culminaron efectivamente en adjudicación (`licitacion_adjudicada == True`), evitando distorsiones por procesos desiertos o cancelados.

```
from src.proyecto import tabla_proporciones

tabla_resumen_tamano = tabla_proporciones(df_procesado, "tamano_proveedor")
print("Proporciones por tamaño en procesos adjudicados (denominador: resultados válidos):")
display(tabla_resumen_tamano)
# la misma función sirve para comparar otra variable, sin duplicar el cálculo.
display(tabla_proporciones(df_procesado, "tipolicitacion"))
```

Proporciones por tamaño en procesos adjudicados (denominador: resultados válidos):

	Ganadora	Perdadora	Sin resultado válido	AB	% Ganadora	% Perdadora
TamanoProveedor						
Grande	12575	6255	0	18830	66.78	33.22
Mediana	4982	4001	0	8983	55.46	44.54
Pequeña	1680	1554	0	3234	51.95	48.05
NoClasificado	1803	1576	0	3379	53.36	46.64
Pequeña	1802	4579	0	6381	52.32	47.68
All	26062	17963	0	44025	59.20	40.80

	Ganadora	Perdadora	Sin resultado válido	AB	% Ganadora	% Perdadora
Tipolicitacion						
Licitación Privada Mayor a 1000 UTM	62	10	0	72	86.11	13.89
Licitación Privada Menor a 1000 UTM	1	1	0	2	50.00	50.00
Licitación Privada entre 100 y 1000 UTM	11	1	0	12	91.67	8.33
Licitación Pública Entre 100 y 1000 UTM (LJ)	14696	8904	0	23599	62.26	37.74
Licitación Pública Mayor 1000 UTM (LP)	8066	6328	0	14394	56.04	43.96
Licitación Pública Mayor a 5000 (LQ)	1652	1559	0	3211	51.45	48.55
Licitación Pública Menor a 100 UTM (LI)	1584	1160	0	2744	57.73	42.27
All	26062	17963	0	44025	59.20	40.80

Python 3.7.12 | Anaconda3-2021.06.0 | Shell | C:\Users\ale\OneDrive\Documents\Proyecto\src\proyecto | C:\Users\ale\OneDrive\Documents\Proyecto\src\proyecto | C:\Users\ale\OneDrive\Documents\Proyecto\src\proyecto

6. Validación técnica y verificación del código

Para satisfacer el criterio de Validación técnica y verificación del código, se ejecutan pruebas automáticas que comprueban el comportamiento del pipeline en:

1. **Caso Normal:** Verificación integral de 6 reglas de calidad sobre el dataset real mediante `validar_dataset_procesado`.
2. **Caso Límite (Edge Case):** Comportamiento frente a fechas anómalas 1900 y registros con proveedor NoClasificado.
3. **Caso Excepción 1 (Columna ausente):** Intento de procesar un dataset sin columna obligatoria, verificando captura de `KeyError`.
4. **Caso Excepción 2 (Dataset vacío):** Intento de procesar un DataFrame sin registros, verificando captura de `ValueError`.

```
pruebas_62 = []

# 1. Caso Normal
res_val = validar_dataset_procesado(df_procesado, len(df_real))
pruebas_62.append({
    "caso": "Caso Normal (Dataset Real)",
    "tipo": "Validación integral",
    "resultado": res_val["total"],
    "detalle": f"({res_val['reglas_superadas']}) reglas superadas sobre {res_val['total_valiados']} registros"
})

# 2. Caso Límite
df_limite = pd.DataFrame({
    "tipolicitacion": ["LIC-100-01", "LIC-100-02"],
    "tamano_proveedor": ["NoClasificado", "Micro"],
    "resultado_oferta": ["Ganadora", "Perdadora"],
    "resultado_licitacion": ["Adjudicada", "Adjudicada"],
    "fecha_publicacion": ["2020-01-01 10:00:00", "2020-01-01 11:00:00"],
    "fecha_cierre": ["2020-01-01 10:00:00", "2020-01-01 10:00:00"],
    "resultado_estado_oferta": ["Ganadora", "Perdadora"]
})

df_limite_prov = limpiar_datos_62(df_limite)
assert df_limite_prov["resultado_estado_oferta"].isnull().all()
assert df_limite_prov["resultado_licitacion"] == "NoClasificado"
pruebas_62.append({
    "caso": "Fechas de 1900 y categoría NoClasificado",
    "tipo": "Tratamiento de fechas anómalas y conservación de categorías",
    "resultado": "OK",
    "detalle": "Fecha 1900 neutralizada a NaN y categoría NoClasificado preservada intacta"
})

# 3. Caso Excepción: Columna obligatoria ausente
df_63_resultado = df_limite.drop(columns=["resultado_oferta"])
try:
    limpiar_datos_63(df_63_resultado)
except KeyError as err:
    pruebas_62.append({
        "caso": "Caso Excepción: Columna faltante",
        "tipo": "Manejo de errores de esquema",
        "resultado": "OK",
        "detalle": f"Captura controlada de KeyError ({err})"
    })
else:
    raise AssertionError("Falló en prueba: no se detectó la columna ausente.")

# 4. Caso Excepción: Dataset vacío
df_vacio = pd.DataFrame(columns=df_limite.columns)
try:
    limpiar_datos_64(df_vacio)
except ValueError as err:
    pruebas_62.append({
        "caso": "Caso Excepción: Dataset vacío",
        "tipo": "Manejo de datasets vacíos",
        "resultado": "OK",
        "detalle": f"Captura controlada de ValueError ({err})"
    })
else:
    raise AssertionError("Falló en prueba: no se detectó el dataset vacío.")
```

```

casos['Pérdida de filas'] = base.iloc[:1].copy()
casos['Indicador incorrecto'] = base.copy()
casos['Indicador incorrecto'].loc[0, 'oferta_ganadora'] = False
casos['Plazo incorrecto'] = base.copy()
casos['Plazo incorrecto'].loc[0, 'plazo_cierre_dias'] = 999
negativo = df_limite.copy()
negativo.loc[0, 'fechacierre'] = '2026-02-01'
casos['Plazo negativo'] = limpiar_datos.f2(negativo)
casos['Derivada ausente'] = base.drop(columns=['plazo_cierre_dias'])
casos['Identificador nulo'] = base.copy()
casos['Identificador nulo'].loc[0, 'wolicitudacion'] = None
for nombre, entrada in casos.items():
    try:
        validar_dataset_procesado(entrada, len(df_limite))
    except AssertionError:
        pruebas_f2.append({'Caso': nombre, 'Resultado': 'OK'})
    else:
        raise AssertionError(f'No se detectó: {nombre}')
display(pd.DataFrame(pruebas_f2).fillna(''))
assert all(p['Resultado'] == 'OK' for p in pruebas_f2)

from src.proyecto import tabla_proporciones, normalizar_categorias, convertir_fechas
muestra_tasas = pd.DataFrame({'Grupo': ['A', 'A', 'B', 'C'],
                                'EstadoLicitacion': ['Adjudicada', 'Adjudicada', 'Adjudicada', 'Revocada'],
                                'ResultadoOferta': ['Ganadora', 'Perdedora', 'None', 'Ganadora']})
tasas = tabla_proporciones(muestra_tasas, 'Grupo')
assert tasas.loc['A', '% Ganadora'] == 50
assert tasas.loc['B', 'All'] == 0 and pd.isna(tasas.loc['B', '% Ganadora'])
assert tasas.loc['B', 'Sin resultado válido'] == 1
assert tasas.loc['All', 'All'] == 2 and 'C' not in tasas.index
assert pd.isna(tabla_proporciones(muestra_tasas.iloc[:0], 'Grupo').loc['All', '% Ganadora'])
# Las etapas también admiten columnas distintas de las del proyecto.
aux = pd.DataFrame({'Etiqueta': ['A', 'None'], 'Fecha': ['1900-01-01', '2026-03-01']})
assert normalizar_categorias(aux, ['Etiqueta']).loc[0, 'Etiqueta'] == 'A'
assert pd.isna(convertir_fechas(aux, ['Fecha'], {'Fecha': [1900]})).loc[0, 'Fecha']
print("OK: funciones reutilizadas con otras columnas y grupos sin resultados válidos.")

```

	Caso	Tipo	Resultado	Detalle
0	Caso Normal (Dataset Real)	Validación integral	OK	6 reglas superadas sobre 44,226 registros
1	Fechas de 1900 y categoría NoClasificado	Tratamiento de fechas anómalas y conservación ...	OK	Fecha 1900 neutralizada a NaT y categoría NoCL...
2	Caso Excepción: Columna faltante	Manejo de errores de esquema	OK	Captura controlada de KeyError ("Faltan column...
3	Caso Excepción: DataFrame vacío	Manejo de conjuntos vacíos	OK	Captura controlada de ValueError (No es posibl...

OK: Todas las pruebas de verificación del código fueron superadas exitosamente.

	Caso	Tipo	Resultado	Detalle
0	Caso Normal (Dataset Real)	Validación integral	OK	6 reglas superadas sobre 44,226 registros
1	Fechas de 1900 y categoría NoClasificado	Tratamiento de fechas anómalas y conservación ...	OK	Fecha 1900 neutralizada a NaT y categoría NoCL...
2	Caso Excepción: Columna faltante	Manejo de errores de esquema	OK	Captura controlada de KeyError ("Faltan column...
3	Caso Excepción: DataFrame vacío	Manejo de conjuntos vacíos	OK	Captura controlada de ValueError (No es posibl...
4	Texto str, NA y columna con datos		OK	
5	Fecha no interpretable rechazada		OK	
6	Pérdida de filas		OK	
7	Indicador incorrecto		OK	
8	Plazo incorrecto		OK	
9	Plazo negativo		OK	
10	Derivada ausente		OK	
11	Identificador nulo		OK	

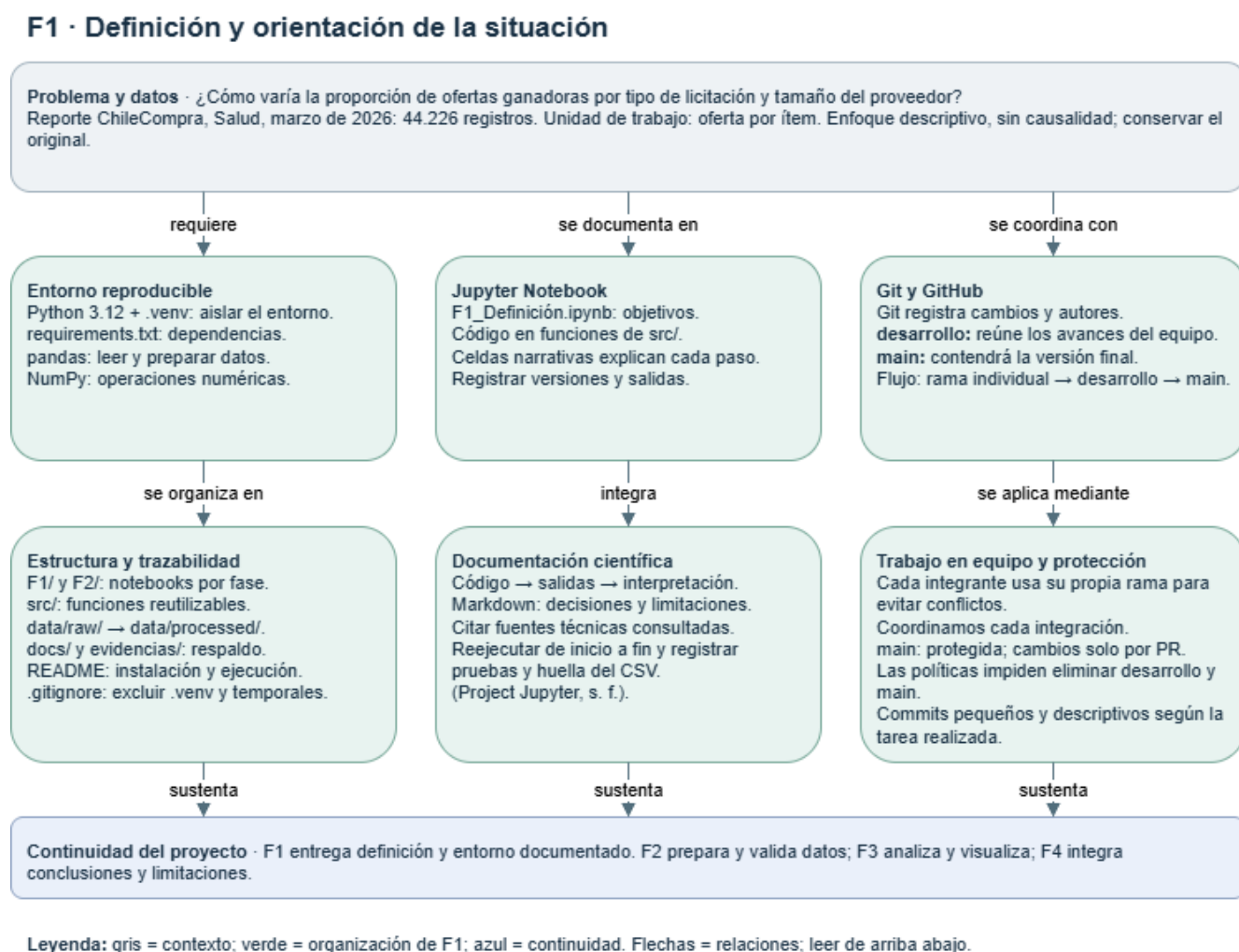
OK: funciones reutilizadas con otras columnas y grupos sin resultados válidos.

VII. Vinculación del mapa conceptual con el avance del proyecto

El mapa conceptual se utilizó como guía para organizar el inicio del proyecto, siguiendo las orientaciones de la Evaluación Formativa 1 (Universidad Andrés Bello, 2026a). A partir del problema y del conjunto de datos se definieron tres componentes principales: el entorno reproducible, el uso de Jupyter Notebook y el trabajo colaborativo mediante Git y GitHub. Estos componentes orientaron la estructura del repositorio, la documentación del proceso y la coordinación de los cambios realizados por el equipo. En esta entrega se implementaron las actividades correspondientes a F1 y F2, mientras que el análisis, las visualizaciones y las conclusiones generales continuarán en F3 y F4.

Figura 1

Mapa conceptual del flujo reproducible inicial del proyecto



Nota. Elaboración propia. La versión editable se encuentra en docs/mcdi500_s1_grupo5.drawio y la versión final en docs/mcdi500_s1_grupo5.pdf.

Tabla 6

Correspondencia entre el mapa conceptual y los productos desarrollados

Componente del mapa	Aplicación en el proyecto	Evidencia
Problema y datos	Se definió la pregunta de análisis, la unidad de trabajo y el alcance descriptivo	F1/F1_Definición.ipynb y data/README.md
Entorno reproducible	Se configuraron Python, el entorno virtual y las dependencias	requirements.txt y README.md
Jupyter Notebook	Se integraron explicaciones, código, resultados y pruebas	Notebooks de F1 y F2
Git y GitHub	Cada integrante trabaja en su rama y los cambios se integran mediante PR	Repositorio e historial de GitHub
Estructura y trazabilidad	Se organizaron datos originales, datos procesados, código y evidencias	data/, src/, docs/ y evidencias/
Documentación científica	Se documentaron decisiones, procedimientos, limitaciones y fuentes	Celdas Markdown, README e informe
Trabajo en equipo y protección	Se definió el flujo rama individual → desarrollo → main	Políticas de ramas y pull requests
Continuidad del proyecto	F2 prepara y valida los datos; F3 y F4 quedan proyectadas	Notebook F2 y planificación del proyecto

VIII. Referencias bibliográficas (Normas APA 7.^a edición)

Las siguientes fuentes técnicas, académicas y docentes respaldan las decisiones descritas en el informe y en los notebooks ejecutados:

ChileCompra. (s. f.-a). *Descargas de datos abiertos del sistema de compras públicas*. Dirección de Compras y Contratación Pública. <https://datos-abiertos.chilecompra.cl/descargas>

ChileCompra. (s. f.-b). *Diccionario de datos de la API de Mercado Público: Licitaciones públicas*. Dirección de Compras y Contratación Pública. <https://www.chilecompra.cl/wp-content/uploads/2026/03/Documentacion-API-Mercado-Publico-Licitaciones.pdf>

ChileCompra. (s. f.-c). *Glosario y definiciones del portal de datos abiertos*. Dirección de Compras y Contratación Pública. <https://datos-abiertos.chilecompra.cl/datos-abiertos/definiciones>

GitHub. (s. f.). *Pull requests*. <https://docs.github.com/en/pull-requests>

McKinney, W. (2022). *Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter* (3.^a ed.). O'Reilly Media.

NumPy Developers. (s. f.). *NumPy user guide and reference documentation* (Versión 2.3). <https://numpy.org/doc/stable/>

Project Jupyter. (s. f.). *Project Jupyter documentation*. <https://docs.jupyter.org/en/latest/>

Samuel, S., & Mietchen, D. (2024). Computational reproducibility of Jupyter notebooks from biomedical publications. *GigaScience*, 13, giad113. <https://doi.org/10.1093/gigascience/giad113>

The pandas development team. (s. f.). *pandas documentation* (Versión 3.0). <https://pandas.pydata.org/docs/>

Universidad Andrés Bello. (2026a). *Evaluación Formativa 1: Mapa conceptual—Exploración guiada de herramientas colaborativas en ciencia de datos* [Instrucciones y rúbrica del curso MCDIA500: Programación para la Ciencia de Datos].

Universidad Andrés Bello. (2026b). *Avance del proyecto Fase 1 y 2: Implementación inicial del entorno reproducible, exploración y procesamiento inicial de los datos del proyecto* [Pauta y rúbrica de evaluación del curso MCDIA500: Programación para la Ciencia de Datos].

IX. Anexos técnicos y trazabilidad

A continuación se resumen los artefactos principales del repositorio que respaldan la reproducibilidad de la entrega:

Artefacto / Archivo	Ruta en Repositorio	Tamaño	Evidencia / Resultado
Dataset crudo original	data/raw/licitaciones_salud_marzo_2026.csv	68,96 MB	SHA-256: 490d9209a10d387011d481b72b7891f26e997974ec2cf9dfc518aa4a08552232
Dataset preprocesado F2	data/processed/licitaciones_salud_marzo_2026_procesado.csv	68,16 MB	SHA-256: 7e835d6725d8c4f9aa64ad97a294dc38fcc341e71373f544c7f6b8ba2ff31dc5
Dataset codificado F2	data/processed/licitaciones_salud_marzo_2026_codificado.csv	69,17 MB	Variables nominales codificadas mediante indicadores 0/1.
Notebook Fase 1	F1/F1_Definición.ipynb	26,6 KB	4 celdas de código ejecutadas; 0 errores.
Notebook Fase 2	F2/F2_Preprocesamiento.ipynb	85,3 KB	10 celdas de código registradas; 0 errores en la última evidencia completa.
Módulo de ingeniería	src/proyecto.py	14,2 KB	Funciones de entorno, lectura, limpieza, codificación, exportación y validación.
Evidencia Fase 1	evidencias/F1_ejecucion.json	0,42 KB	4 celdas ejecutadas, 0 errores, kernel Python 3.12.
Evidencia Fase 2	evidencias/F2_ejecucion.json	0,46 KB	10 celdas ejecutadas, 0 errores, kernel Python 3.12.